

Clothing classification using CNN

1. Objectives

This laboratory is focused on the issue of clothing classification using convolutional neural networks (CNN). By using Python as programming language and some dedicated machine learning platforms (*i.e.*, Tensorsflow with Keras API) the students will develop (from scratch), evaluate and use dedicated deep convolutional neural networks for the clothing classification problem. We show how to develop a robust test framework for estimating the performance of the model, how to improve the model, how to save the model and later load it to make predictions on new data. Finally, the system will be evaluated with respect to objective evaluation metrics such as: accuracy and loss.

2. Theoretical aspects

K-fold cross-validation is a resampling technique employed to determine the performances of machine learning models when dealing with limited data of training examples. The algorithm randomly divides the set of input data into k groups or folds of roughly equal size. The initial fold is designated as validation set and the method is trained on the remaining $k-1$ folds. The procedure has a sole parameter, denoted as k , which indicates the number of groups into which a given data sample is to be divided. Consequently, the procedure is commonly denoted in the technical literature as k -fold cross-validation. When a particular value is chosen for k , it can be substituted in the reference to the model, for instance, $k=10$ becoming 10-fold cross-validation.

Cross-validation is primarily used in applied machine learning to estimate the performance of a machine learning model on data that it has not been exposed to previously. It is a widely used method due to its simplicity and is generally preferred for yielding a less biased or optimistic estimation of the model's performance compared to other techniques, such as a straightforward train/test split.

The general procedure is outlined as bellow:

1. Randomly shuffle the dataset.
2. Divide the input examples into k groups.
3. For each unique group:
 - Select one-fold as a validation data set;
 - Consider the remaining groups as a training dataset;
 - Train a model using the training set and assess its performance on the test set;
 - Retain the evaluation score and discard the model.
4. Summarize the performance of the model using the evaluation scores obtained for all folds.

It is necessary to highlight that each sample from the training dataset is assigned to an individual group and stays in that group throughout the procedure. This implies that each sample has the chance to be included in the holdout set once and is utilized to train the model $k-1$ times.

3. Practical implementation

Pre-requirements: Download the laboratory platform archive from the link provided by the instructor or from Moodle. Extract the archive, then copy the folder containing the .py files into your PyCharm working directory (PycharmProjects). Finally, open PyCharm and load the project using the **Open** option.

Next, select the Python environment “AIenv” that is already created for the project. In PyCharm, go to Settings → Python → Interpreter, then on the right side choose Add Interpreter → Add Local Interpreter... In the window that opens, set Environment to *Select Existing* and locate the “AIenv” environment

Open the FashionRecognition.py python script in the project. The following example presents all the steps necessary to develop from scratch a convolutional neural network (CNN) dedicated for clothing classification. The example uses the Zalando's Fashion-MNIST dataset. The database consists of a training set of 60.000 examples and a testing dataset of 10.000 examples. Each example is a 28 x 28 grayscale image, associated with a label from 10 classes. Zalando intends Fashion-MNIST to serve as a direct drop-in replacement for the original handwritten digit MNIST dataset for benchmarking machine learning algorithms.



Fig. 1. Sample images taken from the Fashion MNIST dataset

Application 1: Considering the Fashion-MNIST dataset, complete the FashionRecognition.py script (using the dedicated machine learning platforms (*i.e.*, Keras API)) in order to be able to recognize the category of an unknown image applied as input. The mapping classes labels (integers from 0 to 9) are as follows: 0 - T-shirt/top, 1 - Trouser, 2 - Pullover, 3 - Dress, 4 - Coat, 5 - Sandal, 6 - Shirt, 7 - Sneaker, 8 - Bag and 9 - Ankle boot. The system performance will be evaluated using the accuracy metric.

Step 1: *Import the dependences necessary to develop Application 1* – The following libraries will be needed in order to develop the script:

```
import numpy as np
from sklearn.model_selection import KFold
from matplotlib import pyplot
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.datasets import fashion_mnist
import cv2
```

Step 2: *Load the Fashion-MNIST dataset in Keras* – The Keras deep learning library provides a convenient method for loading the Fashion-MNIST dataset (`from keras.datasets import fashion_mnist`). The dataset is downloaded automatically the first time the function `fashion_mnist.load_data()` is called and is stored in your home directory. Load the Fashion-MNIST dataset using the following command:

```
(trainX, trainY), (testX, testY) = fashion_mnist.load_data()
```

Step 3: *Visualize the shape of the loaded data* – Determine the number of images existent in the train and test dataset and their size.

Exercise 1: In order to have a graphical description of the Fashion MNIST dataset display the first 9 images existent in the `trainX` variable using the OpenCV library.

Observation 1: As it can be observed the images are all pre-segmented (*i.e.*, each image contains a single item of clothing) and are presented in grayscale format.

Step 4: *Prepare the data* – Write a Python function denoted **prepareData** that takes as input the training and the testing image datasets (`def prepareData(trainX, trainY, testX, testY)`) and process the images as presented in the following sub-steps. The function returns the same set of variables: `trainX, trainY, testX, testY`.

Step 4a: *Reshape the Fashion MNIST dataset* - The Fashion MNIST dataset needs to be reshaped so that it is suitable for CNN training. In Keras, the layers used for two-dimensional convolutions expect input values with the dimensions [samples] [width] [height] [channels]. In the case of RGB, the last dimension (channels) would be 3 for the red, green and blue components and it would be like having 3 image inputs for every color image. In the case of Fashion MNIST, each image in the dataset is a 28 by 28 pixel squares, where the pixel values are gray scale (the number of channels is set to 1).

Step 4b: *Normalize the input values* – The images are grayscale with pixel values between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. This involves first converting the data type from unsigned integers to floats

(`astype('float32')`), then dividing the pixel values by the maximum value. In the case of Fashion - MNIST we can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

Step 4c: *Transform the class labels (a vector of integers) into a binary matrix* – A label (an integer from 0 to 9) is associated to each image that corresponds to the clothing class. This is a multi-category classification problem. It is a good practice to use one hot encoding of the class values, transforming the vector of class integers into a binary matrix. This can easily be done using the built-in `tf.keras.utils.to_categorical()` function.

Step 5: *Define, train and evaluate the model* – The model definition, training and evaluation will be performed within a function denoted `def defineTrainAndEvaluateClassic(trainX, trainY, testX, testY)` that will be called in the main function. The function takes as input the training and testing dataset together with the associated set of labels and finally prints the training accuracy score.

Step 6: *Build the model* – We will construct our model in a function denoted `def defineModel(input_shape, num_classes)`.

Step 6a: *Initialize the sequential model* - We will be using the Sequential model, since our network is a linear stack of layers.

The model has two main aspects: the front end corresponding to the feature extraction (using the convolutional and pooling layers) and the backend represented by the classifier (for data prediction). The layers of the CNN architecture are presented below:

- **Step 6b:** The first hidden layer is a convolutional layer called a **Convolution2D** (Conv2D). The layer has 32 feature maps, with the size of 3×3. This is the input layer, expecting images with the structure outlined above [width][height][channels] (`input_shape=(28, 28, 1)`). The activation function is Relu (`activation='relu'`), while the weights initialization is performed using He initialization (`kernel_initializer='he_uniform'`).
- **Step 6c:** Next we define a pooling layer that takes the max called **MaxPooling2D** of size 2x2. The layer is configured by default with a pool size of 2×2.
- **Step 6d:** Next is a layer that converts the 2D matrix data to a vector called **Flatten**. It allows the output to be processed by standard fully connected layers.
- **Step 6e:** Next a fully connected (**Dense**) layer with 16 neurons to interpret the low level features. The activation function is Relu (`activation='relu'`), while the weights initialization is performed using He initialization (`kernel_initializer='he_uniform'`).
- **Step 6f:** Because we would like to develop a multi-category classification, we know that we will require an output layer with 10 nodes in order to predict the probability distribution of

an image belonging to each of the 10 classes. So, the output layer has 10 neurons for the 10 classes and a *softmax* activation function to output probability-like predictions for each class (`activation='softmax'`).

Step 6g: Compile the model - Finally, we need to compile the model and configure the training process parameters. We decide 3 key factors during the compilation step: the optimizer, the loss function and the metric. For the optimizer a conservative configuration for the stochastic gradient descent will be used with a learning rate of 0.01 and a momentum of 0.9 (`optimizer = optimizer.SGD(learning_rate = 0.01, momentum = 0.9)`). Because the CNN contains *softmax* in the fully connected layer, the system will use as loss function the categorical cross-entropy (`loss = 'categorical_crossentropy'`). As performance metric we will monitor the classification accuracy (`metrics=['accuracy']`).

Step 7: Train the model – Training a model in Keras literally consists only of calling `fit()` function and specifying some training parameters as:

- training data (images and labels),
- validation data (images and labels);
- the number of epochs = 5 (iterations over the entire dataset) to train for;
- the batch size = 32 (number of samples per gradient update).

Step 8: System evaluation – Finally, the test dataset is used to evaluate the model and a classification error rate will be printed.

Step 9: Run the script – Run the script on the computer using CPU. In order to reduce the processing time, train the system also on GPU using the Google Colab platform (<https://colab.research.google.com>). Determine (through coding) the training time in both cases.

Exercise 2: Modify the number of filters in the convolutional layer as specified in Table 1. How is the system accuracy influenced by this parameter? How about the convergence time?

Table 1. System performance evaluation for various numbers of filters in the convolutional layer

Number of filters	8	16	32	64	128
System accuracy					

Exercise 3: Modify the number of neurons in the dense hidden layer as specified in Table 2. The number of filters in the convolutional layer remains set to 32. What can be observed regarding the system accuracy?

Table 2. System performance evaluation for various numbers of neurons in the dense hidden layer

No of neurons	16	64	128	256	512
System accuracy					

Exercise 4: Modify the number of epochs used to train the model as specified in Table 3. The number of neurons in the dense hidden layer is set to 16. What can be observed regarding the system accuracy? How about the convergence time?

Table 3. System performance evaluation for different values of the number of epochs

No of epochs	1	2	5	10	20
System accuracy					

Exercise 5: Modify the learning rate of the stochastic gradient descend as presented in Table 4. Train the model for 5 epochs. What can be observed regarding the system accuracy and the convergence time?

Table 4. System performance evaluation for various learning rates of the SGD

Learning rate	0.1	0.01	0.001	0.0001	0.00001
System accuracy					

Exercise 6: Explore how adding regularization impacts model performance as compared to the baseline model. Add a **Dropout** layer on the CNN (after the **MaxPooling** layer) that randomly excludes 20% of neurons (`model.add(Dropout(0.2))`). Select for the learning rate a value of 0.01. Analyze the convergence speed with and without this regularization operation?

Modify the percentage of excluded neurons in the dropout layer as presented in Table 5. Select a value of 3x3 neurons for convolutional kernel. How is the system accuracy influenced by this parameter?

Table 5. System performance evaluation for number of neurons dropped in the dropout layer

Dropout percentage	0.1	0.2	0.3	0.4	0.5
System accuracy					

Exercise 7: Specify the optimal values for the following parameters (the number of filters in the convolutional layer, the size of the convolutional kernel, the number of neurons in the dense hidden layer, the number of epochs used for training, the learning rate) that maximize the system accuracy.

Exercise 8: Once fit, we can save the final model (architecture and weights) to an *.h5 file by calling the `save()` function on the model and pass in the selected filename (`model.save('Fashion_MNIST_model.h5')`). We can use our saved model to make a prediction on new images. The model assumes that: (1) the new images are in grayscale and have been segmented; (2). contain one centered piece of clothing on a black background and (3) the size of the image is square of 28×28 pixels.

Fig. 2 presents the query image for which the system is required to make a prediction. You can place the image in the working directory with the filename 'sample_image.png'.

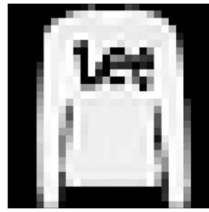


Fig. 2. Query image taken from the testing Fashion MNIST dataset

Using the pre-trained model (saved above) write a Python script able to make an automatic prediction regarding the category of the image presented in Fig.1. For this example, we expect class “2” that corresponds to a “*Pullover*”.

Hint: First, load the image in the working space and convert it to grayscale format. In addition, impose the image size to be 28×28 pixels (as for the other images used for training). This process can be implemented within a `def load_image(filename)` function that will return the reshaped tensor (1, 28, 28, 1), ready for classification purposes.

Importantly, the pixel values are prepared in the same way as the pixel values were prepared for the training dataset when fitting the final model, in this case, normalized. The CNN model saved before can be load using the following function: `model = load_model('Fashion_MNIST_model.h5')`, while the `model.predict(img)` function can be used in order to make a prediction.

Application 2: Consider the example presented in Application 1. The model training has been performed on the entire training dataset (60.000 images). The evaluation has been performed on the testing dataset (10.000 images). In Application 2 a different training strategy will be considered by using three datasets: one for training, one for validation and the final one for testing purposes.

The training dataset can be regarded as a set of examples that are used for learning (data used to fit the model). The validation dataset is a set of examples used to tune the classifier parameters (*ex.* the number of hidden units in a neural network, the kernel of convolution filter ...). More generically, the validation dataset contains data samples used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyper-parameters. The testing dataset represents data samples used to evaluate the final model and to determine the framework performances.

Step 1: Multiple model training – One popular method of creating a validation dataset is to use k-fold cross-validation algorithm. This will divide the training dataset in two parts: one used for the actual training of the model and the other that will be used to tune the model hyperparameters instead of

a separate testing dataset. So, a function denoted `def defineTrainAndEvaluateKFolds(trainX, trainY, testX, testY)` will be developed.

Step 2: Prepare the cross validation datasets – We first need to prepare the parameters of the cross validation data. The **KFold** class from scikit-learn can be used directly in order to split the dataset. The constructor takes as arguments the number of splits, (`k_folds`), whether or not to shuffle the sample data (`shuffle=True`) and the seed (`random_state=1`) for the pseudorandom number generator used prior to the shuffle.

For the current application, we can create an instance that splits a dataset into 5 folds, shuffles prior to the split and uses a value of 1 for the pseudorandom number generator. For 5 folds, the validation dataset is 20% of the training dataset (about 12.000 examples), close to the size of the actual testing set of the problem.

```
kfold = KFold(k_folds, shuffle=True, random_state=1)
```

Step 3: Select data for train and validation for each fold – The training dataset `trainX` will be divided (`kfold.split(trainX)`) in two parts: `trainX_i` (for model training) and `valX_i` (used to perform the model validation). For each possible split (`for train_idx, val_idx in kfold.split(trainX):`):

```
trainX_i = trainX[train_idx]
trainY_i = trainY[train_idx]
valX_i = trainX[val_idx]
valY_i = trainY[val_idx]
```

a model will be defined and trained.

Step 4: Build the model – The same CNN architecture as for Application 1 will be used (`model = defineModel((28, 28, 1), 10)`).

Step 5: Train the model – The baseline model will be trained for 5 training epochs with a default batch size of 32 examples.

```
history = model.fit(trainX_i, trainY_i, epochs=5, batch_size=32,
                    validation_data=(valX_i, valY_i), verbose=1)
```

Step 6: Save the training information – The training process will be repeated recursively for each fold (in the `for` loop) and each model fit (`history`) will be saved within the `histories` list.

Step 7: Fold evaluation – For each fold, the test set will be used to evaluate the model. The accuracy will be determined.

Step 8: Save the accuracy - For each fold, the accuracy obtained at Step 7 is stored in the `accuracyScores` list.

Finally, the function `defineTrainAndEvaluateKFolds` will return two lists of elements: with the trainings information and with the performance accuracy scores.

Step 9: System performance presentation – There are two key aspects that require a particular attention: the diagnostics of the learning behavior during training and the estimation of the model performance. These can be implemented using separate functions.

First, the diagnostics involve creating a line plot showing model performance on the train and validation sets during each fold of the k-fold cross-validation. These plots are important in order to determine if the model is overfitted, underfitted or if it has a good fit for the dataset.

Two figures will be displayed one for loss and the other for the accuracy. Green lines indicate the model performance on the training dataset, while the red lines indicate performance on the hold out validation dataset. The `def summarizeLearningCurvesPerformances(histories, accuracyScores)` function below creates and shows this plot given the collected training histories. Finally, the classification accuracy scores collected during each fold can be summarized by calculating the mean and standard deviation.

```
def summarizeLearningCurvesPerformances(histories, accuracyScores):

    for i in range(len(histories)):

        # plot loss
        pyplot.subplot(211)
        pyplot.title('Cross Entropy Loss')
        pyplot.plot(histories[i].history['loss'], color='green', label='train')
        pyplot.plot(histories[i].history['val_loss'], color='red',
label='validation')

        # plot accuracy
        pyplot.subplot(212)
        pyplot.title('Classification Accuracy')
        pyplot.plot(histories[i].history['accuracy'], color='green',
label='train')
        pyplot.plot(histories[i].history['val_accuracy'], color='red',
label='validation')

    pyplot.show()
```

```
print('Accuracy: mean=%.3f std=%.3f, n=%d' % (mean(accuracyScores) * 100,
std(accuracyScores) * 100, len(accuracyScores)))
```

Exercise 8: Adding padding to the convolutional operation can often result in better model performance, as more of the input image or feature maps are given an opportunity to participate or contribute to the output. By default, the convolutional operation uses ‘*valid*’ padding, which means that convolutions are only applied where possible. This can be changed to `padding='same'` so that zero values are added around the input such that the output has the same size as the input. For Application 1, how is the system accuracy influenced by the padding operation?

Exercise 9: An increase in the number of filters used in the convolutional layer can often improve performance, as it can provide more opportunity for extracting simple features from the input images. This is especially relevant when very small filters are used, such as 3×3 pixels. By applying the padding operation (`padding='same'`) within the convolutional process, increase the number of filters (in the convolutional layer) from 32 to 64. For Application 1, how is the system accuracy influenced by this parameter?

OBSERVATION: The laboratory report (*.pdf file) will contain:

- The code for all the exercises. The code lines introduced in order to solve an exercise will be marked with a different color and will be explained with comments.
- Print screens with the results displayed in the PyCharm console.
- The exercises solutions: tables of results and responses to the questions.